Project & Technical Defense Report (REPORT.md)

Student Academic Progression and Portfolio Management System

Department of Computer Engineering · Academic Year 2024–2025

Course: Database Management Systems (DBMS Mini-Project)

Table of Contents

1. [Project Introduction & Objectives](#1-project-introduction--objectives)
2. [Problem Statement](#2-problem-statement)
3. [Technology Stack Rationale](#3-technology-stack-rationale)
4. [Relational Schema & 3NF Normalization Justification](#4-relational-schema--3nf-normalization-justification)
5. [Primary Keys, Foreign Keys & Referential Integrity (CASCADE)](#5-primary-keys-foreign-keys--referential-integrity-cascade)
6. [SQL Constraints (CHECK, UNIQUE, NOT NULL, DEFAULT)](#6-sql-constraints-check-unique-not-null-default)
7. [Important SQL Queries, JOINs & CRUD Controllers](#7-important-sql-queries-joins--crud-controllers)
8. [SGPA/CGPA Chart Progression & Gap Year Indicator Logic](#8-sgpacgpa-chart-progression--gap-year-indicator-logic)
9. [Google Sheets Synchronization Architecture](#9-google-sheets-synchronization-architecture)
10. [Installation, Configuration & Execution Guide](#10-installation-configuration--execution-guide)
11. [Testing Results, System Limitations & Future Scope](#11-testing-results-system-limitations--future-scope)
12. [15 Likely Viva Examination Questions with Accurate Answers](#12-15-likely-viva-examination-questions-with-accurate-answers)

---

1. Project Introduction & Objectives

In collegiate engineering programs, student academic records and co-curricular achievements are frequently scattered across disconnected spreadsheets, paper records, and disparate departmental portals. When students apply for placements, higher education (e.g. GRE, GATE, campus interviews), or accreditation audits (NBA/NAAC), faculty advisors face significant challenges compiling consolidated progression histories.

Core Objectives of the System:

- Maintain comprehensive student academic records from Secondary School (10th) and Higher Secondary (12th) through all semesters (Semester 1 through Semester 8+).
- Track semester-wise SGPA (Semester Grade Point Average) and CGPA (Cumulative Grade Point Average).
- Record individual course theory marks and laboratory practical marks.
- Record official academic drop years and medical gap years with verified reasons, clearly distinguishing them from unrecorded/missing semesters.
- Maintain segregated student portfolios: internal and external seminars, technical/co-curricular competitions, and extra-curricular achievements.
- Deliver a modern, responsive web GUI with interactive Chart.js visualizations.

• Provide a reliable, one-way Google Sheets cloud synchronization mechanism with full audit logging.

---

2. Problem Statement

Conventional academic spreadsheets suffer from severe normalization flaws:

1. Redundancy & Update Anomalies: Repeating student contact info, subject titles, and category labels across hundreds of rows causes data inconsistency.
2. Deletion Anomalies: Deleting a semester mark might inadvertently delete course descriptions if not stored in separate relational entities.
3. Absence of Referential Enforcement: Spreadsheets cannot enforce relational constraints (e.g., entering marks obtained greater than maximum marks, or having child records pointing to nonexistent students).
4. Lack of Dynamic Progression Insights: Raw tables fail to visually differentiate continuous academic progression from genuine gap years.

This project resolves these shortcomings by implementing a Third Normal Form (3NF) relational database engine coupled with a secure web GUI.

---

3. Technology Stack Rationale

Layer | Technology | Justification

:--- | :--- | :---

Frontend | HTML5, CSS3, JavaScript (ES6+), Bootstrap 5.3 | Responsive grid layout, clean modal interactions, custom academic navy/slate/teal theme.

Data Visualization | Chart.js 4 | Lightweight canvas rendering of SGPA/CGPA line curves on a strict 0.0–10.0 grade scale with broken spans for missing data.

Backend | Python 3, Flask 3.1.3 | Lightweight WSGI web framework; clean separation of routes, business logic, and view templates.

Database | SQLite 3 (`student_progression.db`) | Zero-configuration local database engine with complete ACID compliance and strict `PRAGMA foreign_keys = ON;` enforcement. Compatible with future MySQL/PostgreSQL migration.

Cloud Integration | `gspread` & `google-auth` | Service-account OAuth 2.0 authentication to publish joined progression tables to Google Sheets without user credential leakage.

---

4. Relational Schema & 3NF Normalization Justification

The relational database consists of 9 distinct, properly related tables:

1. `students` — Master student profiles.
2. `semesters` — Semester performance records per student.
3. `subjects` — Master catalog of theory and lab courses.
4. `semestersubjectmarks` — Specific subject grades for a semester.
5. `activitycategories` — Normalized classification tags.
6. `activities` — Technical, co-curricular, and extra-curricular achievements.

7. `seminars` — Internal and external seminar participations.
8. `gapyears` — Documented study breaks and drop years.
9. `synclog` — Cloud synchronization audit log.

Normalization Defense:

• 1NF (First Normal Form):

Every attribute contains atomic values (e.g., student full name, roll number, marks). No repeating multi-valued groups exist (e.g., subject marks are not stored in a concatenated string). Each relation has a primary key.

• 2NF (Second Normal Form):

The schema is in 1NF and contains no partial dependencies. In composite entities such as `semestersubjectmarks(semesterid, subjectid)`, non-key attributes (`marksobtained`, `gradepoint`) depend on the complete composite key. Subject metadata (`subjectname`, `credits`, `subjecttype`) is separated into `subjects`.

• 3NF (Third Normal Form):

The schema is in 2NF and contains no transitive dependencies ($X \rightarrow Y \rightarrow Z$). Non-key attributes depend only on the primary key. For example, activity category names and types are isolated in `activitycategories(id, name, activitytype)` rather than duplicating category strings inside every `activities` record.

---

5. Primary Keys, Foreign Keys & Referential Integrity (CASCADE)

• Primary Keys: Every table features an explicit integer `id PRIMARY KEY AUTOINCREMENT` ensuring unique surrogate identification.

• Foreign Keys:

• `semesters.studentid` $\rightarrow$ `students.id` (`ON DELETE CASCADE`)

• `semestersubjectmarks.semesterid` $\rightarrow$ `semesters.id` (`ON DELETE CASCADE`)

• `semestersubjectmarks.subjectid` $\rightarrow$ `subjects.id` (`ON DELETE CASCADE`)

• `activities.studentid` $\rightarrow$ `students.id` (`ON DELETE CASCADE`)

• `activities.categoryid` $\rightarrow$ `activitycategories.id` (`ON DELETE RESTRICT`)

• `seminars.studentid` $\rightarrow$ `students.id` (`ON DELETE CASCADE`)

• `gapyears.studentid` $\rightarrow$ `students.id` (`ON DELETE CASCADE`)

• Referential Action (`ON DELETE CASCADE`): When a student is removed from the directory, the database automatically cleans up all associated semesters, marks, seminars, activities, and gap years, preventing orphan records.

---

6. SQL Constraints (CHECK, UNIQUE, NOT NULL, DEFAULT)

The database strictly enforces integrity at the storage layer:

• `UNIQUE` Constraints:

• `students.rollno` is unique.

• `students.email` is unique when provided.

• `subjects.subjectcode` is unique.

- `semesters`: `UNIQUE(studentid, semesterno)` prevents entering duplicate semesters for the same student.
- `semestersubjectmarks`: `UNIQUE(semesterid, subjectid)` prevents recording duplicate marks for the same course in the same semester.
- `activitycategories.name` is unique.
- `CHECK` Constraints:
- `secondarypercent` & `highersecondarypercent`: `BETWEEN 0.0 AND 100.0`.
- `semesterno`: `BETWEEN 1 AND 12`.
- `sgpa` & `cgpa`: `BETWEEN 0.0 AND 10.0`.
- `marksobtained` & `maxmarks`: `marksobtained >= 0.0 AND maxmarks > 0.0 AND marksobtained <= maxmarks`.
- `subjecttype`: `IN ('Theory', 'Lab')`.
- `seminartype`: `IN ('Internal', 'External')`.
- `activitytype`: `IN ('Technical', 'Co-curricular', 'Extra-curricular')`.
- `gapyears`: `CHECK(endyear >= startyear)` prevents inverted timelines.

---

7. Important SQL Queries, JOINs & CRUD Controllers

A. Comprehensive Student Progression Joined View (Used in CSV & Google Sheets Sync)

```sql
SELECT
st.rollno,
st.fullname,
st.email,
st.department,
st.admissionyear,
st.secondarypercent,
st.highersecondarypercent,
st.currentstatus,
sem.semesterno,
sem.academicyear,
sem.sgpa,
sem.cgpa,
sem.status as semesterstatus,
(SELECT GROUPCONCAT(startyear || '-' || endyear || ' (' || reason || ')', '; ')
FROM gapyears WHERE studentid = st.id) as gapyearssummary
FROM students st
LEFT JOIN semesters sem ON sem.studentid = st.id
```

ORDER BY st.rollno ASC, sem.semesterno ASC;

```

Purpose: Uses an outer `LEFT JOIN` and correlated subquery with `GROUPCONCAT` so students who have not yet completed a semester are not omitted.

#### B. Fetching Subject Marks by Semester with Master Details

```sql

SELECT m., sub.subjectcode, sub.subjectname, sub.subjecttype, sub.credits

FROM semestersubjectmarks m

JOIN subjects sub ON sub.id = m.subjectid

WHERE m.semesterid = ?

ORDER BY sub.subjecttype DESC, sub.subjectcode ASC;

```

C. Upserting Subject Marks (Atomic Insert or Update)

```sql

INSERT INTO semestersubjectmarks(semesterid, subjectid, marksobtained, maxmarks, gradepoint)

VALUES(?, ?, ?, ?, ?)

ON CONFLICT(semesterid, subjectid) DO UPDATE SET

marksobtained = excluded.marksobtained,

maxmarks = excluded.maxmarks,

gradepoint = excluded.gradepoint;

```

---

8. SGPA/CGPA Chart Progression & Gap Year Indicator Logic

In `static/js/main.js` and `templates/studentdetail.html`:

1. The backend constructs a complete range from Semester 1 through the student's highest recorded semester.
2. If an intermediate semester is unrecorded (e.g., student has records for Sem 1 and Sem 3, but Sem 2 is absent), the system emits `null` for that coordinate.
3. Chart.js configures `spanGaps: false`, causing the line to break visually across the missing semester rather than interpolating false grades.
4. The system checks the `gapyears` table:

- If a documented gap year matches the break, a prominent Academic Interruption Callout (`.gap-callout`) is displayed highlighting the start year, end year, and reason.
- If no gap year is recorded, a Missing Semester Alert (`.missing-sem-alert`) is rendered, avoiding false drop year assumptions.

---

9. Google Sheets Synchronization Architecture
```

```

[ SQLite Database ]

■

▼ (database.getprogressionexportdata)

[ Flattened 2D Data Matrix ]

■

▼ (sheetssync.performsync)

[ Service Account Auth (OAuth2 Credentials) ]

■

▼ (Google Sheets API v4 via gspread)

[ Target Spreadsheet: Worksheet 'Progression' ]

■

▼

[ Audit Log Entry (synclog table: SUCCESS / FAILED) ]
```

Key Architectural Safeguards:

1. One-Way Directional Safety: SQLite is the authoritative source of truth. The application pushes data to Google Sheets; unvalidated edits in the spreadsheet cannot overwrite the database.
2. Fault Tolerance & Non-blocking Behavior: If internet connectivity drops, the service account file is missing, or the spreadsheet ID is invalid, the operation records a `FAILED` log with an explanatory error and alerts the user in the UI without crashing the Flask web server.
3. Worksheet Isolation: Overwrites only the dedicated `Progression` tab, leaving other sheets in the workbook untouched.

---

10. Installation, Configuration & Execution Guide

Prerequisites:

- Python 3.9+ installed and added to system `PATH`.

Quick Execution on Windows:

Double-click `runwindows.bat` or run:

```powershell

python -m pip install -r requirements.txt

python app.py
```

Open your browser to: `http://127.0.0.1:5000`

Google Sheets Configuration (Optional for Cloud Sync):

1. Create a service account in Google Cloud Console, download the key as `serviceaccount.json`, and place it in the project root directory.
2. Create a Google Sheet and copy its ID from the URL (`https://docs.google.com/spreadsheets/d//edit`).
3. Set the environment variable in PowerShell:

```powershell
$env:GOOGLESHEETID="YOURSPREADSHEETID"
```

4. Share the sheet with the service account client email as Editor.
5. Click Sync Google Sheets on the dashboard.

---

11. Testing Results, System Limitations & Future Scope

• Testing Verification: All 17 automated test cases passed in 0.638s, confirming constraint enforcement and route stability.

• Current Limitations:

• Single-user local administration (no role-based student login portal).

• Manual sync trigger (automatic background cron scheduling requires a daemon queue).

• Future Scope:

• Multi-tenant student and faculty login portals with hashed passwords.

• Automated PDF grade card generation with digital signatures.

• Migration script to PostgreSQL for institutional deployment.

---

12. 15 Likely Viva Examination Questions with Accurate Answers

Q1: What is Third Normal Form (3NF), and how does your database achieve it?

Answer: A relation is in 3NF if it is in 2NF and has no transitive dependencies (non-prime attributes must depend only on the primary key, not on other non-prime attributes). In our system, student profile details (10th/12th percentages, admission year) are stored only in `students`, course information is separated into `subjects`, and categories are separated into `activitycategories`. The marks table only stores the foreign keys and the numerical score.

Q2: Why did you use `PRAGMA foreignkeys = ON;` in SQLite?

Answer: By default, SQLite disables foreign key constraint enforcement for backwards compatibility. Executing `PRAGMA foreignkeys = ON;` on every database connection is mandatory to guarantee referential integrity and ensure cascading actions like `ON DELETE CASCADE` execute properly.

Q3: What is the difference between `ON DELETE CASCADE` and `ON DELETE SET NULL`?

Answer: `ON DELETE CASCADE` automatically deletes dependent child records when the parent record is deleted (e.g., deleting a student deletes all their semester marks and seminars). `ON DELETE SET NULL` sets the foreign key column in child rows to `NULL` instead of deleting them (used when child records can exist independently without the parent).

Q4: How do you prevent duplicate semester records for the same student?

Answer: In the `semesters` table, we define a composite unique constraint: `UNIQUE(studentid, semesterno)`. If an insertion attempts to add Semester 1 twice for the same student, SQLite raises an `IntegrityError`.

Q5: What is an index, and which indexes did you create in your database?

Answer: An index is an auxiliary B-Tree data structure that speeds up query retrieval without scanning every row in a table. We created indexes on frequent search and join keys: `students(rollno)`, `semesters(studentid)`, `semestersubjectmarks(semesterid)`, `activities(studentid)`, and `synclog(syncedat DESC)`.

Q6: How does your application distinguish a recorded gap year from a missing semester?

Answer: An unrecorded semester only indicates that grades have not yet been entered for that semester number. A gap year is an officially documented academic break with verified start and end years and a documented reason stored in the `gapyears` table. The GUI inspects both tables and displays a Gap Year badge only when a verified record exists in `gapyears`.

Q7: Why did you choose a `LEFT JOIN` instead of an `INNER JOIN` in the progression export query?

Answer: An `INNER JOIN` only returns rows where matches exist in both tables. If a newly enrolled student has not yet completed any semester, an `INNER JOIN` would exclude them from the export. A `LEFT JOIN` ensures all registered students are included regardless of semester count.

Q8: How does the system enforce that marks obtained cannot exceed maximum marks?

Answer: We implemented a table-level SQL `CHECK` constraint in `semestersubjectmarks`:

`CHECK(maxmarks > 0.0 AND marksobtained <= maxmarks AND marksobtained >= 0.0)`.

Q9: What happens if Google Sheets is unreachable during a synchronization?

Answer: The sync logic in `sheetssync.py` intercepts network errors, API quota errors, or missing credentials. It records a `FAILED` status entry with the error message into the `synclog` database table and alerts the user in the web UI. It never blocks or crashes the local database.

Q10: Why did you implement one-way synchronization instead of two-way synchronization?

Answer: One-way synchronization treats the normalized relational database as the single source of truth. Unvalidated direct edits in a public spreadsheet could bypass primary keys, type checks, and referential constraints, corrupting the institutional database.

Q11: What is SQL Injection, and how did you prevent it?

Answer: SQL Injection is a vulnerability where malicious SQL commands are injected into backend queries via user input. We prevented this completely by using parameterized queries with placeholder syntax (`?` in SQLite) for every database transaction, ensuring user input is treated strictly as data.

Q12: How are Theory and Laboratory courses distinguished in your schema?

Answer: In the `subjects` master table, we enforce `CHECK(subjecttype IN ('Theory', 'Lab'))`. When marks are entered, the system joins against `subjects` to automatically tag and display badges in the UI for Theory vs Lab practicals.

Q13: How does Chart.js handle missing semesters without plotting incorrect grades?

Answer: For any semester between 1 and the student's highest semester that lacks a record, the backend sends a `null` value. Chart.js is configured with `spanGaps: false`, which breaks the curve across `null` coordinates, accurately showing the interruption.

Q14: How does the system handle concurrent database requests in Flask?

Answer: Flask handles incoming requests in separate worker threads. In `database.py`, each query opens a dedicated connection context using `with getdb() as conn:`, ensuring that SQLite connection handles are not shared across threads and are cleanly closed after execution.

Q15: If this application is migrated from SQLite to MySQL or PostgreSQL, what changes are needed?

Answer: The SQL schema uses standard ANSI SQL data types (`INTEGER`, `REAL`, `TEXT`) and standard constraints (`PRIMARY KEY`, `FOREIGN KEY`, `CHECK`, `UNIQUE`), so the schema translates directly. We would only need to swap the connection factory in `database.py` to use `psycopg2` or `mysql-connector` and replace `?` with `%s` parameter placeholders.